SOFTWARE SPECIFICATION

RealEstate

Ghana land & property marketplace — Frontend & Backend

Purpose: hand-off document for the backend team. Specifies the existing Next.js frontend and the production backend required to support it, integrating through a versioned API contract.

Version: 1.0 · **Status:** For implementation

Repositories: Frontend (exists) & Backend (to build) — independently deployable.

RealEstate — Software Specification (Frontend + Backend)

Version: 1.0 · **Status:** For implementation · **Audience:** Backend team, frontend team, DevOps, QA **Owner (product):** RealEstate · **Prepared for:** Backend engineering handover

> This document specifies the **existing** RealEstate frontend and the **backend that must be built** to support it. The frontend is complete and runs today against a browser-based mock service layer; every data access already goes through a typed seam (`src/lib/api/*` returning `src/types` shapes). The backend's job is to satisfy that same contract with a real, production system. Frontend and backend are developed in **separate repositories** and integrate **only through the API contract** defined here (Section 12).

Table of contents

1. Executive summary

What it is. RealEstate is a **trust-first land & property marketplace for Ghana**. Buyers browse land **directly on a live satellite map**, tap the exact plots they want, and buy through a **mobile-money / card → escrow** flow. Sellers list land through a guided wizard and earn a **Verified** badge via a document verification pipeline. The platform also includes a **building-materials & tools shop** (so a buyer can buy land and then build on it), a **land-conflict / boundary-overlap checker** (does my boundary clash with a registered plot?), a **service-provider directory** (surveyors, developers, etc.), buyer↔seller **messaging**, **notifications**, and an **admin** back office (verification queue, moderation, analytics, reports).

Who it is for. Land buyers (diaspora and local), land sellers (individual owners, agents, developers), material suppliers, service providers, and platform administrators.

Business problem. Land fraud, double-selling, and undocumented boundaries are endemic in the Ghanaian land market. RealEstate reduces risk by combining (a) **map-precise plot selection** with authoritative geometry, (b) **document verification** with a visible trust badge, (c) **escrow-based settlement** so money is only released as documents/title transfer, and (d) a **boundary-conflict check** that flags overlaps before money changes hands.

Main capabilities. Auth & roles · listings + search + map plot picker · estate/parcel GeoJSON · escrow purchases + simulated-then-real payments · KYC/verification · messaging + notifications (real-time) · materials catalog + cart + orders + supplier self-service · providers + reviews + leads · land-conflict checker + emailed report · admin analytics/moderation/reports.

High-level architecture. A **Next.js 16** frontend (App Router, TypeScript) talks over HTTPS to a **stateless REST API** backed by **PostgreSQL + PostGIS** (relational + geospatial), **Redis** (cache, queues, rate-limits, presence), **object storage** (images, documents, GeoJSON), a **background-worker** tier (BullMQ) for email/escrow/notifications, a **realtime gateway** (WebSocket) for chat and live notifications, and third-party integrations for **payments** (mobile money + card), **email** (transactional), and optional **SMS**.

Why frontend and backend are separated. The two are built by different teams on different release cadences, in different languages/toolchains, and deployed to different platforms (frontend to an edge/CDN host, backend to a containerized runtime near the database). Decoupling via a **versioned API contract** lets both teams work in parallel, test independently, and deploy independently without reading each other's source.

---

2. Goals and non-goals

Business goals

- Reduce buyer risk (fraud, double-sale, boundary disputes) enough to earn trust and transaction volume.
- Monetize via transaction fees (escrow), verification, featured listings, materials margin, and paid land-conflict checks for guests.

Product goals

- A buyer can go from "browse map" → "own plot with documents" without leaving the platform.
- A seller can list, get verified, and manage leads/sales self-service.
- A guest or seller can check a boundary for conflicts and receive a professional report.

Technical goals

- Backend fully satisfies the existing frontend contract (`src/types` + `src/lib/api/*`) so the mock layer is swapped for real HTTP with **no UI rewrite**.
- Correct, auditable **money** (escrow ledger) and **land geometry** (PostGIS) — the two trust-critical domains.
- Independent build/test/deploy for each repo; contract-tested integration.

Success criteria

- Frontend, pointed at the real API base URL, passes its existing e2e flows (buy flow, seller estate-publish).
- Escrow ledger reconciles to zero; no orphaned held funds.
- Conflict check on server (PostGIS) returns the same overlaps as the client reference within tolerance.
- p95 read-API latency ≤ 400 ms; realtime message delivery p95 ≤ 2 s.

In first release (MVP)

Accounts/roles · listings + search + map + parcels · escrow purchase with **one** live payment provider (plus sandbox) · verification/KYC · messaging + notifications · materials shop + orders + supplier CRUD · land-conflict check + email report · admin queue/moderation/analytics.

Explicitly excluded from first release

Native mobile apps · multi-language UI beyond English (scaffolding exists) · in-platform mortgage/financing · automated registry sync (manual/ops-assisted at launch — see open question OQ-3) · seller payouts automation beyond a manual/queued payout (see OQ-2) · AI valuation.

Possible future capabilities

Registry/Lands-Commission integration · mortgage partners · SMS-first flows · offline map tiles · title-NFT / blockchain audit trail · multi-currency (diaspora) · agent CRM.

---

3. Assumptions, constraints, and open questions

Confirmed requirements (from the built frontend)

- Roles: `buyer`, `seller`, `provider`, `admin` (a seller may also supply materials).
- Prices in **Ghana Cedis (₵, GHS)**, integer-major-unit today; see NFR money rule.
- Payment methods surfaced: `mtn-momo`, `vodafone-cash` (Telecel Cash), `card`.
- Escrow steps: `funds-held → documents-transfer → title-handover → released`.
- Parcels are **GeoJSON polygons** with `ParcelProperties` (plotNumber, owner, status, area, price…).
- Data shapes are frozen by `src/types/index.ts` (Section 11.3 mirrors them).

Assumptions made by the architect (labelled; confirm before build where starred)

- **A1** Backend is a **separate service** (not Next.js route handlers) exposing REST/JSON. *(★ OQ-1)*
- **A2** Stack: **NestJS + TypeScript**, **PostgreSQL 16 + PostGIS**, **Prisma**, **Redis + BullMQ**, **S3-compatible** storage, **native WebSocket gateway**, **Resend** for email. *(★ OQ-1)*
- **A3** Auth is **email+password** with short-lived **JWT access + rotating refresh**; phone-OTP is a fast-follow. *(★ OQ-4)*
- **A4** Escrow at launch is a **platform-operated ledger with manual release approval**, not a regulated third-party escrow. *(★ OQ-2 — legal/financial, must confirm.)*
- **A5** The platform DB is the **source of truth** for "registered parcels"; conflict checks carry a legal disclaimer and are **advisory**, not an official title search. *(★ OQ-3.)*
- **A6** Money stored in **minor units (pesewas) as integers**; the current frontend treats price as whole cedis — backend returns integer major units to preserve the contract and adds a `priceMinor` only if agreed.
- **A7** Guests may run **paid** conflict checks; a guest is identified by a server-issued check token.

Technical constraints

- Frontend is **Next.js 16 / React 19 / TS strict**; it consumes JSON and expects the field names in Section 11.3.
- Geospatial correctness requires PostGIS (or equivalent). Client uses lng/lat rings (GeoJSON order).
- CSP on the frontend host forbids arbitrary third-party script; any browser-side SDK (e.g., a payment widget public key) must be explicitly allowlisted.

Business constraints

- Ghana **Data Protection Act, 2012 (Act 843)** applies to PII (names, phone, IDs, documents, plot ownership).
- Escrow and mobile-money settlement are regulated; the merchant-of-record and any trust-account partner must be confirmed before real money moves (OQ-2).

Information still requiring clarification (open questions)

#	Question	Blocks	Default assumption if unanswered
OQ-1	Backend stack & repo model (separate service vs Next route handlers; NestJS ok?)	Repo setup	A1/A2
OQ-2	Real escrow (licensed partner/trust account) vs platform ledger? Merchant of record? Refund policy? Payout method to sellers?	Payments/escrow build	A4
OQ-3	Is platform DB the source of truth for parcels, or must we integrate the Lands Commission / a registry?	Conflict checker legitimacy	A5
OQ-4	Auth method(s): email+password only, Google OAuth, phone+OTP (SMS gateway)? MFA for admins?	Auth build	A3
OQ-5	Payment provider(s): Paystack, Flutterwave, MTN MoMo API, Telecel — which, and live at launch?	Payments	Paystack + sandbox
OQ-6	Transactional email provider + sending domain you own (SPF/DKIM/DMARC)?	Email	Resend + noreply@ your domain
OQ-7	Object storage (S3 / GCS / Cloudflare R2)? Document encryption/virus-scan required?	Storage	S3 + AV scan on upload
OQ-8	Realtime transport: self-hosted WebSocket vs Pusher/Ably?	Chat/notifications	Self-hosted WS
OQ-9	Notification channels: in-app only, + email, + SMS (which gateway)?	Notifications	in-app + email
OQ-10	Hosting/region & data residency (PII in Ghana/EU/…)?	Deploy	EU/af-region container host
OQ-11	Expected scale (users, listings, checks/day) at launch and 12 months?	Sizing	10k users, 5k listings, 200 checks/day
OQ-12	Keep exact `src/types` field names, or may backend introduce a versioned v2 shape?	Contract	Keep exact shapes

> Per the guideline, these open questions do **not** block the spec: the starred assumptions above are used as defaults and clearly labelled. Confirm OQ-2 and OQ-3 before any real money or legal claim ships.

---

4. User roles and permissions

Role	Description	Key allowed actions	Restricted from
Guest (unauthenticated)	Anonymous visitor	Browse listings/materials/providers; view map & parcels; run **paid** land-conflict check; register/login	Buying land; messaging; dashboards; unpaid checks
Buyer	Registered purchaser	All guest reads; favorites & saved searches; message sellers; start purchases & escrow; buy materials; track orders; free-tier conflict checks (policy)	Creating listings; verification actions; admin
Seller	Lists land; may also supply materials	Buyer actions; create/edit/pause/delete **own** listings & estates/parcels; submit verification; manage **own** leads; CRUD **own** materials; view **own** sales stats	Editing others' data; admin/moderation; approving own verification
Provider	Service provider (surveyor, developer, …)	Buyer actions; manage **own** provider profile; receive quote requests; accrue reviews	Listing/verifying land; admin
Admin	Platform operator	Full read; run verification queue (approve/reject/request docs); moderate listings; manage users; resolve abuse reports; view platform analytics; **approve escrow releases**	— (highest privilege; actions audit-logged & MFA-gated)

Authentication requirements. Guests: none. All others: valid session (JWT access + refresh). Admin actions: MFA required (OQ-4). **Authorization rules.** Ownership-scoped for seller/provider resources (a subject may only mutate rows where `ownerId === subject.id`); role-gated for admin routes; resource-level checks enforced on the **backend** regardless of any frontend guard (the frontend guard `RequireRole` is UX only).

Permissions matrix (C=create, R=read, U=update, D=delete, A=action)

Resource	Guest	Buyer	Seller	Provider	Admin
Listings	R	R	C R U D (own)	R	R U D, moderate (A)
Estates / parcels	R	R	C R U (own)	R	R U
Parcel status	R	R	U (own)	R	U
Purchases / escrow	—	C R (own), A:advance	R (own, as seller)	—	R all, **A:release**
Payments	—	A:pay (own)	—	—	R, A:refund
Verification cases	—	—	C R (own)	—	R all, A:approve/reject/request-docs
Messages / conversations	—	C R (own)	C R (own)	C R (own)	R (moderation)
Notifications	—	R U (own)	R U (own)	R U (own)	R U (own), C (broadcast)
Materials	R	R	C R U D (own)	R	R U D
Material orders	—	C R (own), A:advance*	R (own sales)	—	R all
Providers	R	R	—	C R U (own)	R U D
Reviews	R	C (own)	C (own)	C (own)	R D (moderation)
Leads	—	(creates implicitly)	R U (own)	R U (own)	R
Abuse reports	C	C	C	C	R U A:resolve
Land-conflict check	A:paid	A	A	A	A, R history
Admin analytics/users/reports	—	—	—	—	R U A

* Order status advancement is fulfilment-driven; in production, order status transitions are owned by the supplier/ops, not the buyer (the current buyer-advance affordance is a demo shortcut — see FR-MAT-4).

---

5. User journeys and use cases

Each journey lists **Actor · Preconditions · Trigger · Main flow · Alternative · Failure · Result**, and splits **FE/BE** responsibilities.

UJ-1 Buy land on the map (signature flow)

- **Actor:** Buyer. **Preconditions:** authenticated; estate parcels exist. **Trigger:** taps ≥1 available plot → **Buy**.
- **Main:** select plots (FE running totals) → checkout (choose MoMo/card) → **BE** creates purchase, initiates payment, on success marks plots `sold`, opens **escrow** (`funds-held`) → buyer watches escrow tracker → admin/ops advance `documents-transfer → title-handover`, admin **releases** → purchase `completed` with downloadable documents.
- **Alternative:** buyer selects sold/reserved plot → FE blocks; concurrent buyers race for same plot → **BE** atomic reserve wins, loser gets `409 PLOT_UNAVAILABLE` .
- **Failure:** payment declined → purchase `failed`, plots released, funds not captured.
- **Result:** Buyer owns plots; parcels `sold` ; escrow ledger holds funds until release.
- **FE:** selection store, checkout UI, escrow tracker, purchases list. **BE:** atomic reservation, payment init/verify (webhook), escrow ledger + state machine, document generation, notifications.

UJ-2 Sell land: publish estate + get verified

- **Actor:** Seller. **Trigger:** completes listing wizard (draw grid / upload GeoJSON-KML).
- **Main: BE** creates listing + estate + parcels (validated polygons) → listing `active` (or `pending-review`) → seller submits verification with documents → admin reviews → **verified** flips listing badge; seller notified.
- **Failure:** invalid/overlapping upload → `422` with per-feature errors. **Result:** buyable estate on the map.
- **FE:** wizard, map draw, uploads. **BE:** GeoJSON validation, geometry storage, verification state machine, doc storage.

UJ-3 Check a boundary for conflicts

- **Actor:** Guest (paid) or Seller (policy). **Trigger:** draws/uploads boundary → **Run check**.
- **Main: BE** runs PostGIS overlap against registered parcels → returns `ConflictResult` (searcher ring, conflicts with overlap geometry + area, total, clear flag) → FE overlays blue/amber/red → buyer requests **email report** → **BE** queues transactional email (PDF/HTML) + in-app

notification.

- **Alternative (guest):** must pay one-off fee first → **BE** issues a check token gating the run.
- **Failure:** self-intersecting boundary → `422 INVALID_POLYGON`. **Result:** advisory conflict report (disclaimer).
- **FE:** draw/upload, map overlays, email form. **BE:** payment gate, PostGIS intersect/area, email + notification.

UJ-4 Buy building materials

- **Actor:** Buyer. **Main:** browse catalog → cart → checkout (delivery address + MoMo/card) → **BE** creates order (`processing`) → fulfilment advances `confirmed → dispatched → delivered`; buyer tracks. **BE** owns stock/price snapshot at order time. **Result:** delivered order; supplier sales visible to the supplier.

UJ-5 Messaging & notifications

- **Actor:** Buyer/Seller. **Main:** buyer opens a listing → **Message seller** → **BE** creates conversation + lead → realtime delivery to seller → unread counters → notifications. **Failure:** WS down → FE falls back to poll `GET /conversations`. **Result:** threaded chat; seller lead created.

UJ-6 Admin verification & moderation

- **Actor:** Admin. **Main:** open queue → open case → approve/reject/request-docs (with note) → **BE** updates case + listing badge + timeline + notifies seller; moderate flagged listings; resolve abuse reports.

---

6. Functional requirements

Grouped by module. Each: **role · input · processing · output · errors · owner (FE/BE) · acceptance**. IDs are stable.

Auth & accounts (FR-AUTH)

ID	Title	Role	Input → Processing → Output	Key errors	Owner	Acceptance
FR-AUTH-1	Register	Guest	{name,email,phone,password,role∈{buyer,seller,provider}} → validate, hash (argon2id), create user, issue tokens → `{session}`	409 EMAIL_TAKEN, 422	BE (FE form)	New user can log in; password never stored plaintext
FR-AUTH-2	Login	Guest	{email,password} → verify → tokens → `{session}`	401 BAD_CREDENTIALS, 423 LOCKED	BE	Correct creds return session; 5 fails → lockout
FR-AUTH-3	Refresh	any	refresh cookie → rotate → new access	401	BE	Rotating refresh; reuse detection revokes family
FR-AUTH-4	Logout	any	revoke refresh	—	BE	Token unusable after
FR-AUTH-5	Current user	any	access → `{user}`	401	BE	Returns `User` shape
FR-AUTH-6	Password reset	Guest	request(email) → email token; confirm(token,newpw)	400 EXPIRED	BE	Time-boxed single-use token
FR-AUTH-7	Update profile	owner	patch profile fields → `{user}`	422	BE (FE settings)	Persisted; avatar upload via storage

Listings & discovery (FR-LST)

ID	Title	Role	Notes	Owner
FR-LST-1	Search/list	any	Filters: q, region, landStatus, min/maxPrice, min/maxAcres, amenities[], verification, sellerType, sort∈{newest,price-asc,price-desc,size-desc,most-viewed}, page → `Paged<Listing>`	BE
FR-LST-2	Get one	any	by id → `Listing` ; 404 if missing/removed	BE
FR-LST-3	Create	Seller	full listing (+optional estate/parcels) → `Listing` ; new listings may enter `pending-review` (policy)	BE
FR-LST-4	Update/pause	Seller(own)	patch; lifecycle transitions active↔paused, draft→pending-review	BE
FR-LST-5	Delete	Seller(own)/Admin	soft-delete (`removed`); parcels detached	BE
FR-LST-6	Record view	any	increment views (deduped per session/IP)	BE
FR-LST-7	Featured	any	curated/most-viewed subset	BE
FR-LST-8	Similar	any	by region/price/type near a listing	BE
FR-LST-9	Seller listings	Seller(own)/Admin	by sellerId	BE
FR-LST-10	Regions	any	distinct region list for filters	BE

Estates & parcels — geospatial (FR-GEO)

ID	Title	Role	Notes
FR-GEO-1	List estates	any	`Estate[]` (id,name,center,zoom,listingId)
FR-GEO-2	Estate parcels	any	`ParcelCollection` (GeoJSON) for an estate
FR-GEO-3	Get parcel	any	by id → `ParcelFeature`
FR-GEO-4	Create estate w/ parcels	Seller	accept drawn grid or uploaded GeoJSON/KML; **validate** each polygon (closed, non-self-intersecting, min area, within region bounds); reject overlaps within the same estate; honor feature props (plotNumber, price, status)
FR-GEO-5	Set parcel status	Seller(own)/system	available↔reserved↔sold; sold is terminal except admin correction; status change is **atomic** with purchase

Purchases, payments & escrow (FR-BUY)

ID	Title	Role	Notes
FR-BUY-1	Start purchase	Buyer	{plotIds[], paymentMethod} → **atomically reserve** plots, create `Purchase(processing)`, init payment intent → `{purchase, paymentClientData}`; 409 if any plot unavailable
FR-BUY-2	Payment webhook	system	provider → verify signature (idempotent) → on success capture, mark plots `sold`, purchase→ `in-escrow` with `funds-held`; on fail purchase→ `failed`, release plots
FR-BUY-3	List purchases	Buyer(own)/Admin	`Purchase[]`
FR-BUY-4	Get purchase	Buyer(own)/Admin	`Purchase` incl. escrow steps + documents
FR-BUY-5	Advance escrow	Admin/ops	move `documents-transfer` → `title-handover`; **release** (admin) → `released`, purchase `completed`, funds ledgered out; notify buyer/seller
FR-BUY-6	Monitor toggle	Buyer(own)	`monitored` flag for alerts
FR-BUY-7	Refund	Admin	on dispute/failed handover → provider refund + ledger reversal

Verification / KYC (FR-VER)

ID	Title	Role	Notes
FR-VER-1	Submit case	Seller	{listingId, documents[]} → `VerificationCase(submitted)` + timeline
FR-VER-2	List/queue	Admin	filter by status
FR-VER-3	Get case	Admin/owner	full case + documents (signed URLs) + checks
FR-VER-4	Review action	Admin	approve→ `verified` (listing badge verified) / reject→ `rejected` / request-docs→ `docs-requested`; append timeline + adminNote; per-check pass/fail; notify seller

Messaging & notifications (FR-MSG)

ID	Title	Role	Notes
FR-MSG-1	List conversations	owner	`Conversation[]` with unreadBy
FR-MSG-2	Start conversation	Buyer	{listingId, sellerId, body} → conversation + first message + **lead**
FR-MSG-3	List messages	participant	paged `Message[]`
FR-MSG-4	Send message	participant	body → `Message`; realtime fan-out; update lastMessage/unread
FR-MSG-5	Mark read	participant	zero unread for subject
FR-MSG-6	Unread count	owner	integer badge
FR-NOTE-1	List notifications	owner	`AppNotification[]`
FR-NOTE-2	Mark read / all read	owner	update flags
FR-NOTE-3	Push (system)	system/Admin	create notification + realtime + optional email/SMS

Materials shop (FR-MAT)

ID	Title	Role	Notes
FR-MAT-1	Catalog	any	filters q, category, region, sort∈{popular,price-asc,price-desc,rating} → `Paged<Material>`
FR-MAT-2	Get material	any	by id → `Material`
FR-MAT-3	Place order	Buyer	{lines[], deliveryAddress, region, paymentMethod} → snapshot price/unit, compute delivery fee + total, pay, create `MaterialOrder(processing)`
FR-MAT-4	Advance order	Supplier/ops	`processing→confirmed→dispatched→delivered` (fulfilment-owned)
FR-MAT-5	List orders	Buyer(own)/Supplier(sales)	`MaterialOrder[]`
FR-MAT-6	Supplier CRUD	Seller/Supplier(own)	create/update/delete own `Material` (from `MaterialInput`); appears in public catalog
FR-MAT-7	Supplier products	owner	list own materials

Providers, reviews, leads (FR-PRV)

ID	Title	Role	Notes
FR-PRV-1	List/search providers	any	by category/region → `ServiceProvider[]`
FR-PRV-2	Get provider	any	by id
FR-PRV-3	List reviews	any	by targetId/targetType
FR-PRV-4	Add review	Buyer/authed	{targetId,targetType,rating 1–5,body}; one per subject per target; recompute rating/reviewsCount
FR-PRV-5	Seller leads	Seller(own)	`Lead[]` + status U (new→contacted→qualified→closed)
FR-PRV-6	Seller stats	Seller(own)	aggregates for dashboard

Land-conflict checker (FR-CNF)

ID	Title	Role	Notes
FR-CNF-1	Run check	Seller/Buyer; Guest(paid)	{ring:number[][]} → PostGIS overlap vs registered parcels → `ConflictResult`; ignore slivers < 1 m^2; sort by overlap desc
FR-CNF-2	Guest payment gate	Guest	one-off fee → issue check token required by FR-CNF-1
FR-CNF-3	Email report	any who ran	{result/checkId, name, email} → build report, **queue** email, notify if authed → `{ok, reference}`
FR-CNF-4	Check history	authed/Admin	list prior checks (optional MVP+)

Admin (FR-ADM)

ID	Title	Role	Notes
FR-ADM-1	Platform stats	Admin	users, listings, GMV, escrow held, checks, etc.
FR-ADM-2	Users	Admin	list/search/suspend/role-change
FR-ADM-3	Moderation listings	Admin	list flagged; moderate (approve/remove/flag)
FR-ADM-4	Abuse reports	Admin	list; resolve/dismiss/investigate

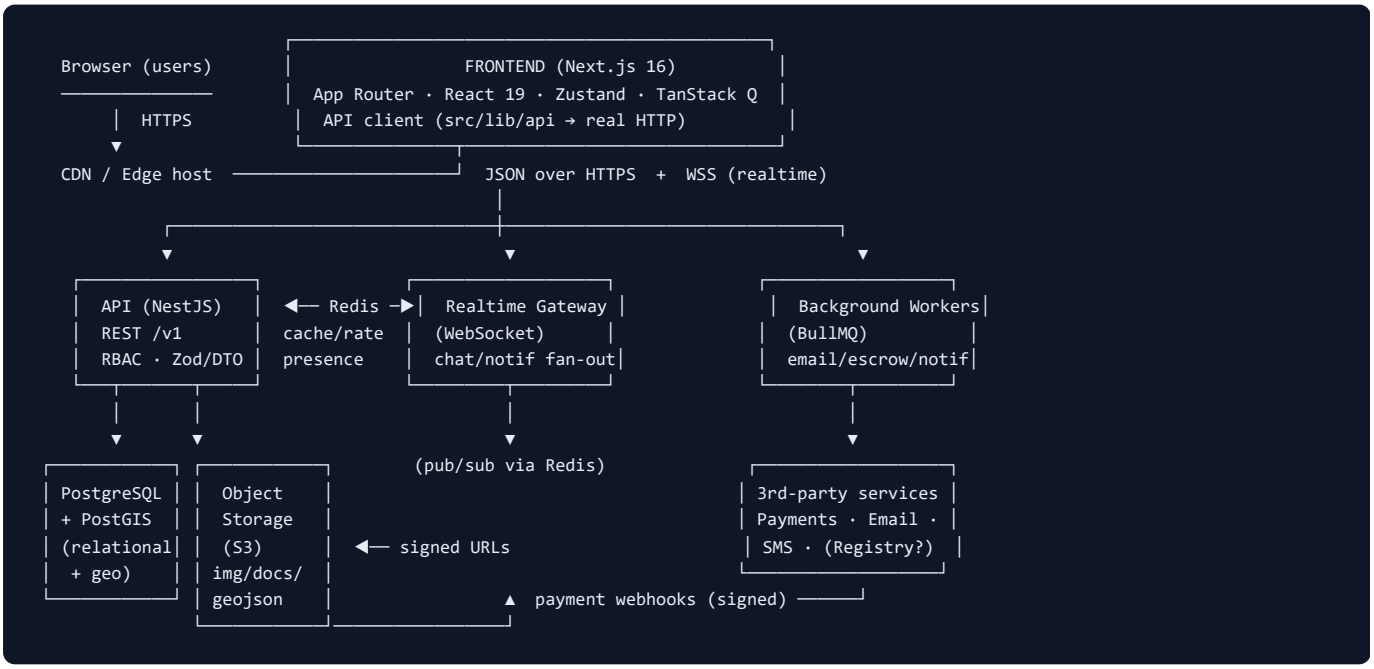
Global acceptance criteria (apply to all FRs): validated input; correct authz; documented success + error responses; idempotency on money/mutation endpoints; audit-logged privileged actions; contract test green.

7. Non-functional requirements

ID	Area	Requirement (measurable)
NFR-1	Performance	Read APIs p95 ≤ 400 ms, p99 ≤ 800 ms at MVP load; conflict check ≤ 2 s p95
NFR-2	Realtime	Message/notification delivery p95 ≤ 2 s; reconnect ≤ 5 s
NFR-3	Availability	99.9% monthly for API; graceful degradation if WS/email/payment down
NFR-4	Scalability	Stateless API horizontally scalable; workers scale by queue depth; DB read replicas for search
NFR-5	Reliability	Money & escrow operations are ACID + idempotent; no lost/duplicated captures; queue retries w/ DLQ
NFR-6	Security	OWASP ASVS L2; TLS 1.2+; argon2id passwords; RBAC + resource ownership; rate limits; signed webhooks
NFR-7	Privacy	Act 843 + GDPR-style: data minimization, right-to-erasure workflow, PII access logged, documents access-controlled
NFR-8	Data retention	Transactions/audit retained ≥ 7 years (land legal); soft-delete elsewhere; deletion honors legal holds
NFR-9	Money representation	Amounts as **integer minor units (pesewas)** internally; API returns whole-cedi integers to match current contract (see A6/OQ-12); currency `GHS`
NFR-10	Time	All timestamps **ISO-8601 UTC**; frontend localizes to Africa/Accra
NFR-11	Rate limiting	Per-IP + per-user token buckets; stricter on auth, checks, reviews, reports; `Retry-After` on 429
NFR-12	Accessibility	WCAG 2.1 AA (frontend); see Section 25
NFR-13	Browser/mobile	Evergreen Chrome/Safari/Firefox/Edge; responsive ≥ 360px
NFR-14	Observability	Structured logs w/ correlation IDs; metrics; traces; error tracking; no PII/secrets in logs
NFR-15	Backup/recovery	Daily DB backup + PITR; object-store versioning; documented RPO ≤ 24 h, RTO ≤ 4 h
NFR-16	Localization	English at launch; i18n keys ready (frontend `next-intl`); currency/number formatting server-neutral
NFR-17	Maintainability	Typed contract, generated client, ≥ 80% backend line coverage on domain logic
NFR-18	File limits	Images ≤ 8 MB (jpg/png/webp); documents ≤ 20 MB (pdf/jpg/png); GeoJSON/KML ≤ 5 MB; AV-scanned

8. High-level system architecture

Textual diagram



Components

- **Frontend (Next.js):** SSR/CSR UI; owns rendering, client state, map; **public**.
- **API (NestJS):** stateless REST `/v1`; validation, RBAC, business rules; **private** behind LB.
- **PostgreSQL + PostGIS:** system of record; geometry + relational; **private**.
- **Redis:** cache, rate-limit buckets, WS pub/sub, presence, BullMQ queues; **private**.
- **Object storage (S3):** images, documents, GeoJSON; served via **signed URLs**; **private buckets**.
- **Realtime gateway:** authenticated WebSocket; chat + notification delivery; **public endpoint, token-gated**.
- **Workers:** email send, escrow transitions, notification fan-out, analytics rollups, AV scan; **private**.
- **Third-party:** payment provider(s), email, SMS, optional registry; reached **server-side only** (secrets on BE).

Data flow (buy)

`FE start-purchase → API reserve+intent → provider pay (client) → provider webhook → API capture+escrow → worker notify → FE polls/receives WS`.

Trust & network boundaries

- **Trust boundary** between browser and API: everything from the client is untrusted; BE re-validates & re-authorizes.
- Only **Frontend**, **API**, **Realtime gateway**, and **payment webhook** endpoints are internet-facing; DB/Redis/storage/workers are in a **private network**. Secrets live only on the backend/worker tier.

---

9. Repository and ownership model

Two independent repos; integrate only via the **shared contract**.

Frontend repository (exists today)

- **Purpose:** the Next.js app users interact with.
- **Structure (actual):** `src/app` (routes), `src/components`, `src/lib/api` (**API client seam**), `src/lib/mock` (removed at go-live), `src/stores` (Zustand), `src/data` (static parcels → replaced by API), `src/types`, `messages/`.
- **Env:** `NEXT_PUBLIC_API_BASE_URL`, `NEXT_PUBLIC_WS_URL`, optional `NEXT_PUBLIC_PAYMENTS_PUBLIC_KEY`, `NEXT_PUBLIC_MAP_TILES_URL` (Section 20).
- **Build/test/deploy:** `npm run build` / `npm test` + Playwright `npm run e2e` / deploy to edge host.
- **API client:** reimplement `src/lib/api/*` over the **generated typed client** from the shared OpenAPI; keep function signatures identical so screens/TanStack Query are untouched.

- **Mock strategy:** current `src/lib/mock` + Prism mock server enable dev before BE is ready; delete mock at cutover.
- **Feature ownership:** all UI, client state, map rendering, accessibility, i18n.

Backend repository (to build)

- **Purpose:** the API, data, jobs, integrations.
- **Structure (recommended):** `src/modules/{auth,users,listings,parcels,purchases,payments,verification,messaging,notifications,materials,providers,reviews,leads,conflicts,admin}`, `src/common` (guards, filters, pipes), `prisma/` (schema+migrations), `test/`, `openapi/`.
- **Env:** DB/Redis/S3 URLs, JWT secrets, provider API keys, email keys (Section 20).
- **Migrations:** Prisma migrate; **PostGIS** enabled via SQL migration. **Workers:** BullMQ processors. **Scheduled:** saved-search alerts, analytics rollups, escrow SLA reminders.
- **Test/deploy:** unit+integration (Testcontainers Postgres/PostGIS) → contract tests → container image → deploy.
- **API docs:** OpenAPI generated from DTOs, published to the shared location + Swagger UI.

Shared artifacts (single source of integration truth)

Artifact	Produced by	Consumed by	Location
OpenAPI spec (`openapi.yaml`, versioned)	Backend	Frontend (client gen), QA (contract tests)	`contracts/` repo or package
JSON Schemas / TS types	Backend (mirror of `src/types`)	Frontend	published npm package `@realestate/contracts`
Event schemas (WS envelopes)	Backend	Frontend	`contracts/events/*.json`
Error-code catalogue	Backend	Frontend, QA	`contracts/errors.md`
Auth & env docs, **API changelog**	Backend	All	`contracts/`

10. Frontend technical specification

10.1 Architecture

Framework **Next.js 16 App Router**; language **TypeScript (strict)**; rendering **hybrid** (static marketing + client-interactive map/dashboards); routing file-based; components **feature-module** folders under `src/components`; **client state** Zustand (persisted: `session`, `favorites`, map `selection`, `cart`); **server state** TanStack Query (caching, retries, invalidation); forms **React Hook Form + Zod**; styling **Tailwind v4 + shadcn/Base UI** (polymorphism via `render` prop); charts **Recharts**; i18n **next-intl** (en complete); error boundaries per route segment; analytics/logging via a thin client wrapper.

10.2 Screens and routes (each: purpose · roles · data · APIs · states)

Route	Screen	Roles	Data / APIs	States (loading/empty/error/success)
`/`	Landing	all	featured listings (FR-LST-7)	skeleton / — / retry / hero+cards
`/listings`	Search	all	FR-LST-1, regions, saved searches	skeleton grid / "no results" / retry / results+pagination
`/property/[id]`	Detail	all	FR-LST-2, similar, reviews, start-conversation	skeleton / — / 404 page / full detail
`/map`	Plot picker	all (buy needs buyer)	estates, parcels (FR-GEO), selection store	map loader / "no parcels" / tile error toast / interactive map
`/checkout`	Land checkout	buyer	selection, FR-BUY-1/2	processing spinner / empty-cart redirect / decline path / receipt+escrow
`/materials`	Materials shop	all	FR-MAT-1	skeleton / "no products" / retry / catalog
`/materials/checkout`	Materials checkout	buyer	cart, FR-MAT-3	processing / empty / decline / order confirm
`/land-check`	Conflict checker	all (guest pays)	FR-CNF-1/2/3	idle hint / no-conflict / invalid-polygon / overlay result
`/service-providers`	Directory	all	FR-PRV-1	skeleton / empty / retry / list
`/dashboard/*`	Buyer dashboard	buyer	purchases, orders, favorites, messages, notifications	per-tab skeleton/empty/error/data
`/seller/*`	Seller dashboard	seller	listings, wizard, leads, verification, materials, messages	idem
`/admin/*`	Admin	admin	stats, queue, moderation, users, reports	idem
`/settings`	Settings	authed	profile/notification prefs	form states
`/login`, `/register[/role]`	Auth	guest	FR-AUTH-1/2	validating / — / field+form errors / redirect to role home

Every data screen implements the four states explicitly; navigation guards via `RequireRole` (UX only; BE enforces).

10.3 Components (selected significant ones)

Layout (nav/footer/role-home routing), shared (Button/Dialog/Toast — Base UI), feature (MapCanvas + selection panel, ListingCard/Grid/Filters, CheckoutStepper + EscrowTracker, ConflictChecker + ConflictMap, MaterialsGrid + Cart, VerificationForm, AdminTables, Charts). For each: typed **props**, local **state**, emitted **events**, **data deps** (which API/query), **validation** (Zod), **a11y** (labels, focus, keyboard), **error handling** (query error → boundary/toast), **reusability boundary** (presentational vs container).

10.4 State model — where each lives

State kind	Home	Example
Local component	`useState`	dialog open, form field focus
Global client	Zustand (persisted)	session, favorites, map selection, cart
Server state	TanStack Query	listings, parcels, purchases, messages
Auth state	Zustand `session` + httpOnly refresh cookie	token, current user
URL state	route/searchParams	listing filters (shareable), map center
Form state	RHF	wizard, checkout, review
Cached	Query cache + persisted stores	last results, unread counts
Persisted	localStorage via Zustand	session, cart, favorites

10.5 API integration layer

Single typed client: base URL from env; **access token** in `Authorization: Bearer` ; **refresh** on 401 via rotating refresh cookie then retry once; request **cancellation** via AbortController (TanStack Query); **retry** idempotent GETs (max 2, backoff), never retry non-idempotent without idempotency key;

timeout 10 s default; **cache invalidation** by query keys on mutations; **pagination** cursor/offset per contract; **uploads** to signed URLs (direct-to-S3) then attach keys; **downloads** via signed URLs; **WS** client with auth handshake + reconnect/backoff; **API version** pinned in base path `/v1`; **error normalization** maps the standard error envelope (Section 13) to typed UI errors.

10.6 Frontend security

Access token in memory (not localStorage) with refresh in **httpOnly SameSite=strict cookie**; React auto-escapes (XSS) + sanitize any HTML (e.g., email preview) via allowlist; **CSRF** mitigated by bearer tokens + SameSite; **CSP** (self + allowlisted map/payment hosts); **route protection** client guard + BE enforcement; **permission-based rendering** hides unauthorized controls; **no secrets** in the bundle (only `NEXT_PUBLIC_*`).

10.7 Frontend testing

Type	Tool	Scope	Coverage goal
Unit	Vitest	utils, stores, api adapters	≥ 80%
Component	Vitest + RTL	cards, forms, checkout	key components
Integration	Vitest + MSW	screen ↔ mocked API	main flows
E2E	Playwright	buy flow, seller publish, conflict check	critical journeys green
Accessibility	axe + Playwright	key pages	0 serious violations
Contract	schemas/Prism	responses match OpenAPI	all consumed endpoints
Visual (opt.)	Playwright snapshots	landing, listing, map	no unintended diffs

---

11. Backend technical specification

11.1 Architecture

Framework **NestJS** (modular, DI); language **TypeScript**; pattern **modular monolith** (feature modules, clear boundaries, extractable to services later); **service layer** for business rules; **repository layer** via Prisma; domain models mirror `src/types`; **background jobs** via BullMQ; **event processing** via Redis pub/sub (WS) + domain events; **caching** Redis; **file handling** S3 signed URLs + AV scan; structured **logging** (pino) with correlation IDs; **config** via env + schema validation at boot.

11.2 Backend modules (each: responsibilities · entities · endpoints · events · jobs · authz)

Module	Owns	Entities	Emits events	Jobs
auth	register/login/refresh/reset, sessions, RBAC	User, RefreshToken, PasswordReset	`user.registered`	reset-email
users	profile, avatars	User	`user.updated`	—
listings	CRUD, search, views	Listing, ListingDocument	`listing.created/updated/removed`	search-index, saved-search-match
parcels	estates, parcels, status, geometry	Estate, Parcel(geom)	`parcel.status_changed`	geojson-validate
purchases	escrow state machine, documents	Purchase, EscrowStep, PurchasePlot, LedgerEntry	`purchase.*`, `escrow.*`	doc-generate, escrow-SLA
payments	intents, capture, refund, webhooks	Payment, WebhookEvent	`payment.captured/failed/refunded`	reconcile
verification	KYC cases + review	VerificationCase, Timeline, Check	`verification.decided`	notify-seller
messaging	conversations, messages	Conversation, Message	`message.created`	fan-out
notifications	in-app + email/SMS	Notification	—	deliver-email, deliver-sms
materials	catalog, orders, supplier CRUD	Material, MaterialOrder, OrderLine	`order.*`	fulfilment-updates
providers	directory	ServiceProvider	—	rating-recompute
reviews	reviews + ratings	Review	`review.created`	rating-recompute
leads	seller leads + stats	Lead	`lead.created`	—
conflicts	overlap check, paid gate, report email	LandCheck, CheckPayment	`check.completed`	build-report-email
admin	analytics, moderation, users, reports	AbuseReport, AuditLog	`moderation.*`	analytics-rollup

11.3 Data model (mirrors `src/types` — field names are the contract)

Key entities and their fields (types abbreviated; all have `id`, audit `createdAt`, and soft-delete `deletedAt` where applicable). Store money as integer minor units internally; expose per NFR-9.

- **User** `{id, name, email(unique), phone, role∈{buyer,seller,provider,admin}, avatarUrl, company?, region, bio?, verified:bool, rating?, reviewsCount?, joinedAt}` (+ private: passwordHash, mfaSecret?).
- **Listing** `{id, title, type∈{land,house,commercial}, landStatus∈{developed,semi-developed,greenfield, undeveloped}, price, negotiable, region, city, address, coords{lat,lng}, sizeAcres, plotsTotal, plotsAvailable, images[], description, amenities[], verification∈{verified,pending,unverified}, sellerId(FK User), sellerType∈ {agent,owner,developer}, estateId?(FK), views, saves, leads, status∈{active,paused,draft,pending-review,flagged, removed}, attributes{...}, documents[ListingDocument], salesAgreement, terms, createdAt}`.
- **ListingDocument** `{id, name, type∈{indenture,site-plan,surveyor-report,id,title-certificate,other}, sizeKb, uploadedAt, verified}` (+ private storageKey).
- **Estate** `{id, name, region, city, center{lat,lng}, zoom, listingId(FK), description}`.
- **Parcel** (GeoJSON feature) props `{id, estateId(FK), plotNumber, owner, status∈{available,reserved,sold}, areaSqm, lengthM, breadthM, price}` + `geometry: Polygon` stored as **PostGIS** `geometry(Polygon,4326)`.
- **VerificationCase** `{id, listingId, listingTitle, sellerId, sellerName, status∈{submitted,under-review, docs-requested,verified,rejected}, documents[], timeline[{status,date,note?}], checks[{label,passed:bool|null}], submittedAt, adminNote?}`.
- **Conversation** `{id, participantIds[], participants[pick(User)], listingId?, listingTitle?, lastMessage, lastMessageAt, unreadBy:Record<userId,count>}`; **Message** `{id, conversationId, senderId, body, sentAt}`.
- **Purchase** `{id, receiptNo, buyerId, plots[PurchasePlot], totalAreaSqm, amount, fees, paymentMethod∈ {mtn-momo,vodafone-cash,card}, status∈{processing,in-escrow,completed,failed}, escrow[EscrowStep{key∈ {funds-held,documents-transfer,title-handover,released}, label, description, status∈{complete,current,pending}, date?}], monitored, documents[{name,type}], createdAt}`; **PurchasePlot** `{parcelId, plotNumber, estateId, estateName, areaSqm, price}`. (+ private **LedgerEntry** for double-entry escrow accounting.)
- **Review** `{id, targetId, targetType∈{agent,provider,listing}, authorId, authorName, authorAvatarUrl, rating 1–5, body, createdAt}`.
- **ServiceProvider** `{id, userId?, name, category∈{surveyor,property-manager,developer,painter,electrician, plumber,photographer}, services[], region, city, rating, reviewsCount, avatarUrl, verified, description, startingPrice, jobsDone, yearsActive}`.
- **AppNotification** `{id, userId, type∈{message,price-drop,verification,escrow,listing,system}, title, body, href?, read, createdAt}`.

- **SavedSearch** `{id, userId, name, filters:ListingFilters, alerts, createdAt}`.
- **Lead** `{id, listingId, listingTitle, buyerId, buyerName, buyerAvatarUrl, kind∈{message,call-request,offer, site-visit}, note?, status∈{new,contacted,qualified,closed}, createdAt}`.
- **AbuseReport** `{id, targetType∈{listing,user}, targetId, targetLabel, reporterName, reason, detail, status∈{open,investigating,resolved,dismissed}, createdAt}`.
- **Material** `{id, name, category∈{cement,blocks,roofing,steel,aggregates,timber,plumbing,electrical,paint, tools,doors-windows,tiles}, brand, price, unit, supplierName, supplierId?, region, rating, reviewsCount, inStock, description, deliveryDays, popular?}`.
- **MaterialOrder** `{id, orderNo, buyerId, lines[{materialId,name,unit,price,qty}], subtotal, deliveryFee, total, paymentMethod, status∈{processing,confirmed,dispatched,delivered}, deliveryAddress, region, createdAt, eta}`.
- **LandCheck** `{id, userId?|guestToken, searcherRing, searcherSqm, conflicts[ParcelConflict{parcelId,plotNumber, owner,estateId,estateName,status,overlapSqm,overlapRings}], totalOverlapSqm, clear, reference?, createdAt}`.

ER relationships (textual): User 1—* Listing (sellerId); Listing 1—0..1 Estate; Estate 1—* Parcel; Buyer(User) 1—* Purchase; Purchase — Parcel (via PurchasePlot); Listing 1—* VerificationCase; User — Conversation; Conversation 1—* Message; User 1—* Notification/SavedSearch/Lead(as buyer); Supplier(User) 1—* Material; Buyer 1—* MaterialOrder; Provider 0..1—1 User; Review *→ (Listing|Provider|User) polymorphic via targetType.

Indexes: Listing(region, price, landStatus, status, createdAt), full-text on title/description/city; Parcel **GiST on geometry** + (estateId,status); Purchase(buyerId,status); Message(conversationId,sentAt); Material(category, region); Notification(userId,read,createdAt); unique(User.email), unique(Review.authorId,targetId,targetType).

Migration considerations: enable PostGIS first; seed from the frontend's mock seed (`src/lib/mock/seed.ts`, `src/data/*`) for parity in staging; new listings default `pending-review` if moderation-on.

11.4 Business rules

ID	Rule	Trigger	Action / exception
BR-1	A plot may be sold once	purchase capture	atomic reserve→sold; concurrent → `409 PLOT_UNAVAILABLE`
BR-2	Funds held until release	capture	ledger credit escrow; only Admin release → debit to seller payable
BR-3	Sold parcels are immutable	status change	reject unless Admin correction w/ audit
BR-4	Verified badge only via approved case	verification approve	set listing.verification=verified + notify
BR-5	One review per subject per target	add review	`409 REVIEW_EXISTS`; recompute aggregates
BR-6	Guest conflict check requires paid token	run check	`402 PAYMENT_REQUIRED` without valid token
BR-7	Overlap slivers < 1 m² ignored	check compute	drop from conflicts
BR-8	Order price is snapshotted	place order	persist unit price at order time (ignore later price change)
BR-9	Seller edits only own resources	any mutate	`403 FORBIDDEN` on ownership mismatch
BR-10	Payment webhooks idempotent	webhook	dedupe by provider event id

11.5 Authentication & authorization

Registration (argon2id) → email verification (optional gate) → login → **JWT access (15 min) + refresh (30 d, rotating, reuse-detected)**; logout revokes refresh; **RBAC** guard by role + **ownership** guard by resource; admin MFA (TOTP); account lockout after 5 failed logins (15 min); sessions tracked server-side by refresh family; all privileged actions **audit-logged**. Auth flow (textual sequence): `Client → POST /auth/login → API verify → {access, set-cookie refresh} → Client calls API with Bearer → on 401 → POST /auth/refresh (cookie) → new access → retry`.

11.6 Background processing

Queues (BullMQ/Redis): `email`, `notifications`, `escrow`, `documents`, `analytics`, `av-scan`, `saved-search`. Retry with exponential backoff (e.g., 5 attempts, 2^n s), **DLQ** for exhausted jobs + alert; jobs **idempotent** (keyed); priorities (payments/escrow > email > analytics); scheduling via repeatable jobs (nightly rollups, saved-search alerts, escrow SLA reminders); bounded **concurrency**; per-queue monitoring + failure alerts.

11.7 Backend security

Input validation (DTO + Zod/class-validator); output serialization (strip private fields via response DTOs); parameterized queries via Prisma (**no SQL injection**); authn (JWT) + authz (RBAC + ownership) on every route; TLS in transit; **encryption at rest** for DB + documents (KMS); secrets in a manager (never in code/logs); rate limiting (Section 11 NFR-11); abuse prevention on checks/reviews/reports; **audit logging** of privileged actions; dependency scanning (npm audit + SCA in CI); **file-upload security** (type/size validation, AV scan, random keys, no execution, signed short-lived URLs); **webhook signature verification** (payments/email); PII handling per Act 843 (access-controlled, minimized, erasable).

11.8 Backend testing

Type	Tool	Scope	Goal
Unit	Jest/Vitest	services, geometry, ledger	≥ 85% domain
Repository	Testcontainers PG+PostGIS	queries, geo overlap	correctness
Integration/API	Supertest	endpoint + authz	all endpoints
Contract	Prism/Dredd vs OpenAPI	responses match schema	all
Migration	CI	up/down + seed	green
Security	ZAP/npm-audit	authz, injection, headers	0 high
Load	k6	search, check, buy	meet NFR-1/2
Worker	Jest	retry/idempotency/DLQ	key jobs

12. API contract

Style: REST/JSON. **Base URL:** `https://api.realestate.example/v1`. **Versioning:** URL major (`/v1`) + deprecation headers. **Content-Type:** `application/json` (uploads via signed URLs / multipart to storage). **Auth:** `Authorization: Bearer <access>` ; refresh via httpOnly cookie. **Correlation:** accept/emit `X-Request-Id`. **Dates:** ISO-8601 UTC. **Money:** integer GHS (see NFR-9), field `amount` / `price`. **Pagination:** `?page=&pageSize=` → `Paged<T>{items,total,page,pageSize,totalPages}` (matches frontend). **Filtering/sorting/search:** query params mirroring `ListingFilters` / `MaterialFilters`. **Idempotency:** `Idempotency-Key` header required on POST purchases/orders/payments. **Rate limits:** `X-RateLimit-*` + `429 Retry-After`. **Deprecation:** `Deprecation` / `Sunset` headers + changelog; no silent breaking change.

Endpoint catalogue (maps 1:1 to `src/lib/api/*`)

Method	Path	Maps to	Roles
POST	`/auth/register`	auth.register	guest
POST	`/auth/login`	auth.login	guest
POST	`/auth/refresh`	—	any
POST	`/auth/logout`	auth.logout	any
GET	`/auth/me`	auth.getSession	authed
POST	`/auth/password/reset-request`	auth	guest
POST	`/auth/password/reset`	auth	guest
GET	`/users/:id`	users.getUser	authed
PATCH	`/users/me`	users.updateProfile	authed
GET	`/listings`	listings.getListings	any
GET	`/listings/featured`	listings.getFeatured	any
GET	`/listings/:id`	listings.getListing	any
GET	`/listings/:id/similar`	listings.getSimilar	any
POST	`/listings`	listings.createListing	seller
PATCH	`/listings/:id`	listings.updateListing	seller(own)
DELETE	`/listings/:id`	listings.deleteListing	seller(own)/admin
POST	`/listings/:id/view`	listings.recordView	any
GET	`/sellers/:id/listings`	listings.getSellerListings	seller(own)/admin
GET	`/regions`	listings.getRegions	any
GET	`/estates`	parcels.getEstates	any
GET	`/estates/:id/parcels`	parcels.getParcels	any
POST	`/estates`	parcels.createEstate	seller
GET	`/parcels/:id`	parcels.getParcelById	any
PATCH	`/parcels/:id/status`	parcels.setParcelStatuses	seller(own)/system
POST	`/purchases`	purchases.startPurchase	buyer
GET	`/purchases`	purchases.getPurchases	buyer(own)/admin
GET	`/purchases/:id`	purchases.getPurchase	buyer(own)/admin
POST	`/purchases/:id/escrow/advance`	purchases.advanceEscrow	admin/ops
PATCH	`/purchases/:id/monitor`	purchases.toggleMonitor	buyer(own)
POST	`/payments/webhook`	(new)	provider
GET	`/verification-cases`	verification.getCases	admin
GET	`/verification-cases/:id`	verification.getCase	admin/owner
POST	`/verification-cases`	verification.submit	seller
POST	`/verification-cases/:id/actions`	verification.review	admin
GET	`/conversations`	messages.getConversations	authed(own)
POST	`/conversations`	messages.startConversation	buyer
GET	`/conversations/:id/messages`	messages.getMessages	participant

Method	Path	Maps to	Roles
POST	`/conversations/:id/messages`	messages.sendMessage	participant
POST	`/conversations/:id/read`	messages.markRead	participant
GET	`/me/unread-count`	messages.getUnread	authed
GET	`/notifications`	notifications.get	authed
POST	`/notifications/:id/read`	notifications.markRead	authed
POST	`/notifications/read-all`	notifications.markAllRead	authed
GET	`/materials`	materials.getMaterials	any
GET	`/materials/:id`	materials.getMaterial	any
POST	`/materials`	materials.createMaterial	supplier
PATCH	`/materials/:id`	materials.updateMaterial	supplier(own)
DELETE	`/materials/:id`	materials.deleteMaterial	supplier(own)
GET	`/suppliers/:id/materials`	materials.getSupplierMaterials	supplier(own)
POST	`/material-orders`	materials.placeOrder	buyer
GET	`/material-orders`	materials.getOrders	buyer(own)/supplier
POST	`/material-orders/:id/advance`	materials.advanceOrder	supplier/ops
GET	`/providers`	providers.getProviders	any
GET	`/providers/:id`	providers.getProvider	any
GET	`/reviews`	reviews.getReviews	any
POST	`/reviews`	reviews.addReview	authed
GET	`/sellers/:id/leads`	leads.getLeads	seller(own)
PATCH	`/leads/:id`	leads.updateLead	seller(own)
GET	`/sellers/:id/stats`	leads.getStats	seller(own)
POST	`/land-checks`	conflicts.checkLandConflict	authed / guest(paid)
POST	`/land-checks/guest-payment`	(new)	guest
POST	`/land-checks/:id/email`	conflicts.sendConflictReportEmail	ran-check
GET	`/admin/stats`	admin.getStats	admin
GET	`/admin/users`	admin.getUsers	admin
GET	`/admin/listings`	admin.getModerationListings	admin
POST	`/admin/listings/:id/moderate`	admin.moderate	admin
GET	`/admin/reports`	admin.getReports	admin
PATCH	`/admin/reports/:id`	admin.resolveReport	admin

Fully-specified example endpoints

POST `/purchases` — start a land purchase. Roles: buyer. Headers: `Authorization`, `Idempotency-Key`. Body:

```
{ "plotIds": ["oyibi-hillcrest-032","oyibi-hillcrest-033"], "paymentMethod": "mtn-momo" }
```

Processing: validate plots exist & `available`; **atomically reserve**; create `Purchase(processing)`; init payment intent. Success `201`:

```
{ "data": { "purchase": { "id":"pur_9x", "receiptNo":"RE-2026-0007", "status":"processing",
  "plots":[{"parcelId":"oyibi-hillcrest-032","plotNumber":"OY-032","estateId":"oyibi-hillcrest",
  "estateName":"Oyibi Hillcrest Gardens","areaSqm":650,"price":42000}], "amount":84000, "fees":1680,
  "paymentMethod":"mtn-momo", "escrow":[{"key":"funds-held","label":"Funds held","status":"pending"}],
  "monitored":false, "documents":[], "createdAt":"2026-07-24T10:00:00Z" },
  "paymentClientData": { "provider":"paystack", "reference":"psk_abc", "authorizationUrl":"https://…" } },
  "meta": { "requestId":"req_123" } }
```

Errors: `409 PLOT_UNAVAILABLE` (with which plotIds), `422 VALIDATION`, `401`. Side effects: plots reserved; `payment.intent.created`. Idempotent by key. **FE:** selection→checkout. **BE:** reservation + payment init.

POST `/land-checks` — run a boundary conflict check. Roles: authed, or guest with `X-Check-Token`. Body: `{ "ring": [[-0.087,5.826], [-0.086,5.826],[-0.086,5.827], [-0.087,5.827],[-0.087,5.826]] }`. Processing: close ring; validate simple polygon; PostGIS `ST_Intersection` / `ST_Area` vs registered parcels; drop <1 m². Success `200`: `ConflictResult` (see 11.3). Errors: `402 PAYMENT_REQUIRED` (guest, no token), `422 INVALID_POLYGON`. Emits `check.completed`.

GET `/listings` — search. Query mirrors `ListingFilters`. `200`: `Paged<Listing>`. No side effects. Cacheable.

> **Deliverable:** the backend team publishes a full **OpenAPI 3.1** document covering every row above with request/ response schemas generated from DTOs; it is the versioned integration artifact (Section 9). This section is its outline.

13. Standard API response formats

Success envelope: `{ "data": <payload>, "meta": { "requestId": "…", "pagination"?: {…} } }`. **Error envelope:** `{ "error": { "code":"PLOT_UNAVAILABLE", "message":"Human readable", "details"?:{…}, "fieldErrors"?:{"email":"Already in use"}, "requestId":"…", "timestamp":"2026-07-24T10:00:00Z" } }`.

Error-code catalogue (excerpt)

Code	HTTP	Meaning	Frontend behavior	Retryable	User message
VALIDATION	422	Bad input	show field errors	no	"Please fix the highlighted fields."
BAD_CREDENTIALS	401	Login failed	show form error	no	"Email or password is incorrect."
UNAUTHENTICATED	401	No/expired token	refresh→retry, else login	once	—
FORBIDDEN	403	Not allowed	toast + hide action	no	"You don't have access to this."
NOT_FOUND	404	Missing resource	404 page/empty	no	"Not found."
PLOT_UNAVAILABLE	409	Plot taken	refresh map, deselect	no	"That plot was just taken."
REVIEW_EXISTS	409	Duplicate review	disable submit	no	"You already reviewed this."
PAYMENT_REQUIRED	402	Guest check unpaid	open pay dialog	after pay	"Pay to run this check."
INVALID_POLYGON	422	Self-intersecting boundary	highlight boundary	no	"Boundary can't cross itself."
RATE_LIMITED	429	Too many requests	backoff + toast	after Retry-After	"Slow down a moment."
PAYMENT_DECLINED	402/200-webhook	Payment failed	show decline path	retry pay	"Payment was declined."
CONFLICT	409	State conflict	refetch	no	—
SERVER_ERROR	500	Unexpected	error boundary + report	yes (idempotent)	"Something went wrong."
SERVICE_UNAVAILABLE	503	Dependency down	degrade + retry	yes	"Temporarily unavailable."

14. Real-time communication

Transport: authenticated **WebSocket** (WSS) with Redis pub/sub fan-out (OQ-8). Fallback: TanStack Query polling if WS unavailable. **Auth:** access token in connect handshake; server validates + subscribes the socket to the user's channels. **Channels:** `user:{id}` (notifications, unread), `conversation:{id}` (messages), `purchase:{id}` (escrow updates). **Envelope:** `{ "event":"message.created", "id":"evt_1", "ts":"…", "data": {…} }`. **Reconnect:** exponential backoff; on reconnect the client **re-fetches** missed state via REST (source of truth), so delivery is at-least-once +

REST reconciliation (no strict ordering guarantee needed). Heartbeats (ping/pong 30 s); duplicate events deduped by `id`; **backpressure** via per-socket send buffer limit; connection limits per user.

Event	Producer	Consumer	Trigger	Payload	Authz	Guarantee
`message.created`	messaging	participants	new message	`Message`	participant only	at-least-once + REST reconcile
`conversation.updated`	messaging	participants	lastMessage/unread change	`{conversationId,lastMessage,unreadBy}`	participant	idem
`notification.created`	notifications	owner	new notification	`AppNotification`	owner	idem
`escrow.updated`	purchases	buyer(+seller)	step change	`{purchaseId, escrow[]}`	owner	idem
`parcel.status_changed`	parcels	map viewers (opt.)	plot sold/reserved	`{parcelId,status}`	public	best-effort
`check.completed`	conflicts	requester	async check done	`{checkId}`	requester	at-least-once

15. Third-party integrations

Credentials stay **server-side**; only an explicitly public payment key may reach the browser (allowlisted in CSP).

Integration	Purpose	Auth	BE responsibility	FE responsibility	Failure behavior	Sandbox
Payments (Paystack/Flutterwave/MoMo — OQ-5)	Charge MoMo/card; escrow funding	secret key + webhook secret	init intent, verify webhook (signature+idempotent), capture/refund, reconcile	render provider widget/redirect with public key	mark purchase failed, release plots, user retry	provider test keys
Email (Resend/ZeptoMail — OQ-6)	Conflict reports, receipts, verification, resets	API key + verified domain (SPF/DKIM/DMARC)	queue+send, handle bounces/complaints via webhook	none (server-sent)	retry queue→DLQ; user still sees in-app	provider sandbox
SMS (optional — OQ-9)	OTP, delivery/escrow alerts	API key	send, rate-limit	none	fall back to email/in-app	test numbers
Object storage (S3/GCS/R2 — OQ-7)	images, documents, GeoJSON	IAM creds	issue signed upload/download URLs, lifecycle, AV scan	upload direct via signed URL	block on scan fail; retry	local MinIO
Map tiles (Esri/OSM)	satellite + street basemap	tile URL/key	proxy/allowlist if keyed	render via MapLibre	show street fallback	public tiles
Land registry (optional — OQ-3)	authoritative parcel truth	TBD	sync/verify parcels; attach official refs	show "official" badge	advisory-only disclaimer	TBD

16. Validation rules (catalogue excerpt)

Backend is the **final authority**; frontend mirrors for UX. Each: type · required · bounds · format · normalize · error code · message.

Field	Type	Rules	Error
email	string	required, RFC email, lowercased, ≤254	VALIDATION/ `email`
password	string	required, ≥10, ≥1 letter+number, not breached-common	VALIDATION/ `password`
phone	string	required, Ghana `+233` /0-format normalized to E.164	VALIDATION/ `phone`
role (register)	enum	one of buyer/seller/provider	VALIDATION/ `role`
listing.price	int	required, ≥ 0, ≤ 1e9 (GHS)	VALIDATION/ `price`
listing.title	string	required, 5–120	VALIDATION/ `title`
coords	obj	lat −90..90, lng −180..180; within GH bounds (soft)	VALIDATION/ `coords`
parcel ring	number[][]	≥4 points, closed, non-self-intersecting, area ≥ 10 m²	INVALID_POLYGON
review.rating	int	1–5	VALIDATION/ `rating`
order.qty	int	1–9999	VALIDATION/ `qty`
deliveryAddress	string	required, 5–200	VALIDATION/ `deliveryAddress`
upload (image)	file	jpg/png/webp, ≤8 MB	VALIDATION/ `file`
upload (doc)	file	pdf/jpg/png, ≤20 MB, AV clean	VALIDATION/ `file`
geojson/kml	file	valid geometry, ≤5 MB, ≤500 features	INVALID_POLYGON

17. Error and failure handling

Situation	Backend returns	Frontend does
Validation failure	422 + fieldErrors	inline field errors, keep form
Auth failure	401	refresh once → else redirect to login
Authorization failure	403	toast; hide action
Not found	404	404 page / empty state
Duplicate/conflict	409 (+code)	contextual message; refetch
Rate limited	429 + Retry-After	backoff, disable, toast
Server error	500 (requestId)	error boundary + "report" with requestId
Database failure	503	retry idempotent; maintenance notice
External-service failure (pay/email)	502/503 or webhook-later	show pending/decline; queue continues
Network/timeout	client-side	retry idempotent GET; offline banner
Partial failure (order line OOS)	409 with details	show which line; let user adjust
WebSocket disconnect	—	reconnect + REST reconcile
Stale data / version mismatch	409 or `Deprecation`	refetch; prompt refresh on major version change

18. API mocking and parallel development

Contract-first: backend drafts **OpenAPI** → both teams review → committed to `contracts/`. Frontend develops against (a) the **existing** `src/lib/mock` layer and (b) a **Prism** mock server generated from OpenAPI; static fixtures come from the current seed data. A **generated typed client** + shared TS types package keep FE/BE in sync. **Contract tests** (Dredd/Prism) validate the real backend against the spec in CI. Auth is **stubbed** in mock mode; **mock WebSocket** emits sample events. A shared **integration environment** hosts the latest backend. **Contract lock:** the OpenAPI version is frozen per release at the start of the QA phase; changes after lock go through Section 19.

19. API change-management process

The **backend team owns** the API contract. Changes are proposed via PR to `contracts/` with a rationale and a FE-impact note; reviewed by a FE + BE representative. **Breaking changes** (removed/renamed field, type change, new required input, semantic change) require a **major version bump** (`/v2`) and a migration window; additive changes are minor. Deprecated fields are marked (`Deprecation` / `Sunset` headers + changelog), kept for **≥ 1 release / 60 days**, then removed. Every change updates the **API changelog** and must pass **contract tests**; releases are coordinated so no backend change **silently breaks** the frontend.

---

20. Environment and configuration

Environments: **local**, **test/CI**, **shared dev**, **staging**, **production**. Never commit real secrets.

Frontend env

Var	Purpose	Req	Example (non-secret)	Secret
`NEXT_PUBLIC_API_BASE_URL`	REST base	yes	`https://api-staging.realestate.example/v1`	no
`NEXT_PUBLIC_WS_URL`	WebSocket	yes	`wss://api-staging…/ws`	no
`NEXT_PUBLIC_PAYMENTS_PUBLIC_KEY`	client pay widget	if card widget	`pk_test_…`	no (publishable)
`NEXT_PUBLIC_MAP_TILES_URL`	basemap	yes	Esri/OSM template	no
`NEXT_PUBLIC_ENV`	env label	yes	`staging`	no

Backend env

Var	Purpose	Req	Secret
`DATABASE_URL`	Postgres+PostGIS	yes	yes
`REDIS_URL`	cache/queues	yes	yes
`JWT_ACCESS_SECRET` / `JWT_REFRESH_SECRET`	token signing	yes	yes
`S3_ENDPOINT` / `S3_BUCKET` / `S3_KEY` / `S3_SECRET`	storage	yes	yes
`PAYMENTS_SECRET_KEY` / `PAYMENTS_WEBHOOK_SECRET`	payments	yes	yes
`EMAIL_API_KEY` / `EMAIL_FROM` / `EMAIL_DOMAIN`	transactional email	yes	key: yes
`SMS_API_KEY`	OTP/alerts	optional	yes
`CORS_ORIGINS`	allowed FE origins	yes	no
`APP_BASE_URL`	links in emails	yes	no
`RATE_LIMIT_*` , `LOG_LEVEL` , `SENTRY_DSN`	ops	optional	dsn: yes

---

21. Deployment architecture

Frontend

Host: edge/CDN platform (e.g., Vercel/Cloudflare). Build `npm run build` → hybrid static+server output; env vars per environment; **preview deployments** per PR; CDN caching for static + `stale-while-revalidate` ; rollback to prior immutable deployment; custom domain + TLS.

Backend

Host: containerized runtime near the DB (e.g., managed containers/K8s). Docker image for **API** + separate **worker** process + **WS gateway** (can co-deploy or split); managed **PostgreSQL+PostGIS**, **Redis**, **S3**; run **migrations** as a pre-deploy step (gated); **health checks** (`/health/live` , `/health/ready`); horizontal autoscale on CPU/RPS (API) and queue depth (workers); **rollback** to previous image + backward-safe migrations; **zero-downtime** via rolling deploy + expand/contract migrations.

Independent compatibility: because integration is the versioned API, frontend and backend deploy on their own cadence; the contract's backward-compatibility policy (Section 19) guarantees a new backend serves the current frontend and vice-versa within a major version.

22. CI/CD specification

Per repo pipeline stages: install → format check → lint → type-check → unit → integration → security scan → **contract test** → build → artifact (image/deployment) → deploy (staging) → smoke test → (manual gate) → deploy (prod) → smoke. **Branching:** trunk-based with short-lived PR branches; protected `main`; release tags. Backend adds **migration test** + **Testcontainers** integration; frontend adds **Playwright e2e** + **axe**. Failing contract tests **block** merge.

23. Observability and operations

Structured JSON logs (pino) with **correlation/request IDs** propagated FE→API→worker; log levels; **metrics** (RPS, latency histograms, error rate, queue depth, escrow-held total, checks/day) via Prometheus/OpenTelemetry; **distributed tracing**; **error tracking** (Sentry) FE + BE; uptime + synthetic checks on key journeys; DB & Redis & queue dashboards; **audit logs** for privileged/admin/money actions; alert thresholds (error rate > 2%, p95 > target, DLQ > 0, escrow reconciliation mismatch, payment webhook failures). **No PII/secrets in logs.**

24. Performance and scalability

Initial ~10k users / 5k listings / 200 checks/day (OQ-11); plan 10× headroom. Targets per NFR-1/2. DB: proper indexes (Section 11.3), **GiST** for geometry, read replicas for search, connection pooling; **caching** (Redis) for hot reads (featured, regions, provider lists); **CDN** for static + images (via storage/CDN); cursor pagination for large lists; **lazy-load** map tiles/parcels by viewport; **code-splitting** per route (Next); query optimization (avoid N+1 via Prisma includes/dataloader); **horizontal scaling** stateless API; **worker scaling** by queue depth; load-test scenarios: search burst, map-parcel fetch, concurrent plot purchase race, conflict-check throughput, checkout+webhook.

25. Accessibility and responsive design

WCAG **2.1 AA**: full keyboard navigation (incl. map controls have accessible alternatives/list view), visible focus, managed focus in dialogs/wizard, screen-reader labels on all controls, associated form labels + inline error announcements (`aria-live`), color contrast ≥ 4.5:1 (chart colors are CVD-validated), and **reduced-motion** support (Framer Motion respects `prefers-reduced-motion`). Breakpoints: mobile ≥ 360px (single-column, bottom nav), tablet (two-column), desktop (full map + panels). Map interactions degrade to a **list/table** fallback for non-pointer users.

26. Testing and QA strategy

Test type	Team	Environment	Tool	Trigger	Coverage goal	Blocking
FE unit/component	FE	CI	Vitest+RTL	PR	≥80%	yes
FE e2e	FE	CI/staging	Playwright	PR/nightly	critical journeys	yes
FE a11y	FE	CI	axe	PR	0 serious	yes
BE unit	BE	CI	Jest/Vitest	PR	≥85% domain	yes
BE repo/geo	BE	CI (Testcontainers)	PG+PostGIS	PR	correctness	yes
BE API/integration	BE	CI	Supertest	PR	all endpoints	yes
Contract	both	CI	Prism/Dredd	PR	all endpoints	yes
Load	BE/DevOps	staging	k6	pre-release	meet NFRs	non-block (gate)
Security	Sec	CI/staging	ZAP/SCA	PR/nightly	0 high	yes (high)
Cross-repo E2E	QA	staging	Playwright → real API	pre-release	buy, publish, check, order, verify	yes

Cross-repo E2E scenarios verify both repos together (e.g., register→list→verify→buy→escrow-release→documents).

27. Team responsibility matrix (RACI)

R=Responsible, A=Accountable, C=Consulted, I=Informed. Teams: **FE**, **BE**, **DevOps**, **Design**, **QA**, **Sec**, **Prod**.

Area	FE	BE	DevOps	Design	QA	Sec	Prod
Product requirements	C	C	I	C	C	I	A/R
UX design	C	I	I	A/R	C	I	C
Frontend architecture	A/R	C	C	C	I	C	I
Backend architecture	C	A/R	C	I	I	C	I
Database schema	I	A/R	C	I	I	C	I
API schema (contract)	C	A/R	I	I	C	C	C
Authentication	C	A/R	C	I	C	A/R	I
Authorization	C	A/R	I	I	C	R	I
API client	A/R	C	I	I	C	I	I
Mock server	R	C	I	I	C	I	I
Deployment	C	C	A/R	I	I	C	I
Monitoring	C	R	A/R	I	I	C	I
Security	C	R	C	I	C	A/R	I
End-to-end testing	C	C	C	I	A/R	C	C
Production incidents	R	R	A/R	I	C	C	I
API changes	C	A/R	I	I	C	C	C

28. Integration workflow

Stages with **entry** → **exit** criteria:

1. **Requirements approved** — entry: this spec; exit: Prod sign-off.
2. **API contract drafted** — entry: requirements; exit: OpenAPI in `contracts/`.
3. **Both teams review contract** — exit: FE+BE approval recorded.
4. **Contract committed** (OpenAPI + event + error schemas) — exit: published `@realestate/contracts`.
5. **FE builds on mock/Prism** — exit: screens function against mock.
6. **BE implements contract** — exit: endpoints live in dev.
7. **Contract tests validate BE** — exit: green in CI.
8. **FE client regenerated/updated** — exit: FE points at real API.
9. **Integration testing** — exit: cross-repo E2E green in staging.
10. **Staging validated** — exit: perf/security gates pass, Prod sign-off.
11. **Coordinated production release** — exit: smoke green, monitoring nominal.

29. Implementation phases (no time estimates)

Phase	Objective	BE tasks	FE tasks	Shared	Deliverable / acceptance
0 Foundation	repos, contract, infra	scaffold NestJS, DB+PostGIS, CI, OpenAPI skeleton	contract package, Prism mock wiring	`contracts/`, env docs	mock server + client generate; CI green
1 Auth	identity & roles	register/login/refresh/reset, RBAC, MFA(admin)	swap session store to real auth	auth docs	login works end-to-end; guards enforced
2 Listings + Geo	browse & map	listings CRUD+search, estates/parcels, GeoJSON validate	point listings/map screens at API	parcel schema	map buyable; search filters correct
3 Purchases + Payments + Escrow	transact	reserve, payment intent+webhook, escrow ledger, docs	checkout + escrow tracker on API	payment sandbox	buy flow + escrow release; ledger reconciles
4 Verification	trust	KYC cases + review + badge	verification screens	doc storage	approve flips badge; notifies seller
5 Messaging + Notifications + Realtime	engage	conversations/messages, notifications, WS gateway	chat + bell on API/WS	event schemas	realtime chat; unread accurate
6 Materials	build	catalog, orders, supplier CRUD	shop/cart/orders on API	—	order lifecycle works
7 Providers + Reviews + Leads	ecosystem	directory, reviews, leads, seller stats	provider + seller dashboards	—	reviews recompute; leads flow
8 Conflict checker	boundary safety	PostGIS overlap, paid gate, report email	checker screens on API	check schema	server overlap matches reference; email sends
9 Admin	operate	stats, moderation, users, reports	admin screens on API	—	queue + moderation live
10 Integration/QA/Deploy/Post-launch	ship & harden	contract tests, load, security, runbooks	e2e, a11y, perf	release coord	staging validated; prod launched; monitored

Suggested order: 0→1→2→3 unlock the signature journey first; 4–9 layer on; 10 throughout and at the end.

Risks per phase: payments/escrow legality (P3), geometry correctness (P2/P8), realtime scaling (P5).

30. Product backlog (sample; priorities: Must/Should/Could/Future)

Backend

ID	Epic	Item	Priority	Acceptance
BE-1	Auth	JWT + rotating refresh + RBAC	Must	FR-AUTH-1..5 pass
BE-2	Geo	PostGIS parcels + GeoJSON validation	Must	FR-GEO-4/5
BE-3	Buy	atomic reserve + escrow ledger	Must	BR-1/2, FR-BUY-1..5
BE-4	Pay	provider intent + signed webhook	Must	FR-BUY-2, BR-10
BE-5	Verify	case state machine + badge	Must	FR-VER-*
BE-6	Realtime	WS gateway + Redis fan-out	Should	Section 14 events
BE-7	Conflict	PostGIS overlap + paid gate + email	Must	FR-CNF-*
BE-8	Materials	orders + supplier CRUD	Should	FR-MAT-*
BE-9	Admin	stats/moderation/reports	Should	FR-ADM-*
BE-10	Notif	in-app + email delivery	Should	FR-NOTE-*

Frontend

ID	Epic	Item	Priority
FE-1	Client	reimplement `src/lib/api/*` over generated client	Must
FE-2	Auth	real session + refresh + guards	Must
FE-3	Realtime	WS client + reconcile	Should
FE-4	Uploads	direct-to-S3 signed uploads	Must
FE-5	Cleanup	remove `src/lib/mock` + RoleSwitcher	Must

Shared / DevOps / QA

ID	Item	Priority
SH-1	OpenAPI + contracts package + Prism mock	Must
SH-2	Error-code catalogue + event schemas	Must
DO-1	Infra (PG+PostGIS, Redis, S3, container host)	Must
DO-2	CI/CD both repos + contract gate	Must
DO-3	Observability (logs/metrics/traces/alerts)	Should
QA-1	Cross-repo E2E suite	Must

---

31. Definition of ready

A task is ready when: it has an ID and acceptance criteria; the relevant **contract (endpoint/event/schema)** is drafted; roles/authz are specified; validation rules are known; UX/empty/error states are defined (FE) or data model/migration is known (BE); dependencies are identified; test approach is agreed.

32. Definition of done

Implementation complete; code reviewed; unit/integration tests pass; **API contract verified** (contract test green); security checks pass (authz, input, secrets); accessibility checked (FE); monitoring/metrics added (BE); docs/changelog updated; acceptance criteria confirmed; feature behind flag if partial.

---

33. Risks and mitigations

ID	Risk	Prob	Impact	Team	Mitigation	Contingency	Owner
RK-1	Escrow/payments legal & regulatory (OQ-2)	Med	High	BE/Prod	confirm partner/merchant-of-record early; ledger + audit	sandbox-only until compliant	Prod
RK-2	Conflict checker treated as official title search (OQ-3)	Med	High	BE/Prod	prominent advisory disclaimer; registry integration path	disable paid claim	Prod
RK-3	API contract drift between repos	Med	Med	Both	contract tests block CI; versioned OpenAPI	freeze + hotfix	BE
RK-4	Auth mismatch (token/refresh/CORS)	Med	Med	BE/FE	shared auth doc + integration env early	rollback	BE
RK-5	Env/config mismatch across stages	Med	Low	DevOps	schema-validated env; per-stage docs	—	DevOps
RK-6	Backend delays block FE	Med	Med	Both	mock/Prism parallel dev	ship FE against mock	Prod
RK-7	FE mock data differs from real shapes	Low	Med	Both	seed from same fixtures; contract tests	fix adapters	FE
RK-8	Breaking API change ships silently	Low	High	BE	Section 19 policy + deprecation headers	major version	BE
RK-9	Realtime scaling/ordering issues	Med	Med	BE	REST reconcile; dedupe; backpressure	poll fallback	BE
RK-10	Deployment incompatibility (FE↔BE versions)	Low	Med	DevOps	backward-safe migrations; version pin	rollback	DevOps
RK-11	Third-party (pay/email) outage	Med	Med	BE	queue+retry+DLQ; degrade gracefully	manual reconcile	BE
RK-12	Geometry correctness (overlap false neg/pos)	Med	High	BE	PostGIS + reference tests vs client algorithm	flag+manual review	BE
RK-13	PII/document exposure (Act 843)	Low	High	Sec	signed URLs, encryption, access logs	breach playbook	Sec
RK-14	Plot double-sale race	Low	High	BE	atomic reserve + unique constraint	reconcile+refund	BE

34. Final implementation checklist

- **Product:** journeys signed off; OQ-1..12 answered; disclaimers approved (escrow, conflict check).
- **Frontend:** `src/lib/api/*` reimplemented over generated client; auth/refresh live; uploads via signed URLs; mock layer & RoleSwitcher removed; e2e + a11y green.
- **Backend:** all endpoints + events implemented; PostGIS geometry + overlap; escrow ledger reconciles; payments webhook signed+idempotent; verification badge flow; migrations + seed; ≥85% domain coverage; OpenAPI published.
- **DevOps:** infra provisioned; CI/CD both repos with contract gate; secrets in manager; backups + PITR; monitoring+alerts.
- **QA:** contract tests green; cross-repo E2E green; load meets NFRs.
- **Security:** authz matrix enforced; rate limits; encryption in transit/at rest; dependency scan clean; PII review.
- **Integration readiness:** staging validated; changelog current; rollback rehearsed.
- **Production launch:** smoke green; escrow reconciliation job running; on-call + runbooks ready.

35. Final architecture decisions (ADRs)

Decision	Selected	Alternatives	Reason	Trade-off / consequence
Backend runtime	NestJS + TypeScript (★OQ-1)	FastAPI/Python, Go, Next route handlers	shared TS types with FE; strong module system; ecosystem	not the fastest raw runtime; team must know TS/Nest
Data store	PostgreSQL + **PostGIS**	Mongo, MySQL, separate geo service	relational integrity + first-class geometry for parcels/overlap	ops must manage PostGIS; migrations need care
ORM	Prisma	TypeORM, Drizzle, raw SQL	DX, typed queries, migrations	raw PostGIS via `$queryRaw` for geo ops
Contract style	REST + OpenAPI 3.1	GraphQL, gRPC	matches existing `src/lib/api` function-per-endpoint seam; easy mocking	over-fetch vs GraphQL; mitigate with field selection where needed
Realtime	Self-hosted WebSocket + Redis (★OQ-8)	Pusher/Ably, SSE, polling-only	control + cost; Redis already present	must operate WS scaling; fallback = polling
Escrow	Platform ledger, admin-released (★OQ-2)	Licensed third-party escrow	ship MVP; auditable double-entry	regulatory risk — must validate legally before real money
Parcel truth	Platform DB, advisory checks (★OQ-3)	Registry integration	no registry API dependency at launch	checks are advisory; add registry later
Payments	Provider abstraction, 1 live at launch (★OQ-5)	Direct MoMo only	flexibility across MoMo+card	more integration surface
Money units	Integer minor units internally, whole-cedi in API (A6/OQ-12)	float, decimal-in-API	no float errors; preserves current contract	conversion layer at the edge
Auth	JWT access + rotating refresh; admin MFA (★OQ-4)	server sessions, OAuth-only	stateless scale + revocation via refresh family	refresh-token store + rotation logic

---

End of specification. Starred (★) items depend on the open questions in Section 3 and must be confirmed with the product owner before the corresponding module is built. This document is versioned; the API contract (OpenAPI) is the binding integration artifact between the two repositories.